CSR101: Python Programming

(AI & Data Engineering Specialization)

Data Structures, Functions, OOP & Modules Handbook

LPU School of Computer Science and Engineering Academic Faculty

Academic Year 2026–2027

Contents

1 Core Syntax, Types, Operators & Built-in Functions Reference **1**
- 1.1 Python Virtual Machine Architecture & Memory Models . . . 1
- 1.2 Primitive Data Types & Representation Limits . . . 1
- 1.3 Operator Precedence Matrix (Highest to Lowest) . . . 2

2 Control Flow, Iteration Protocols, Function Signatures & Regular Expressions **3**
- 2.1 Control Flow Mechanics . . . 3
- 2.2 Function Parameter Anatomy & Argument Unpacking . . . 3
- 2.3 Scope Resolution: The LEGB Rule . . . 4
- 2.4 Regular Expressions Cheat-Sheet (`re` Module) . . . 4

3 Data Structure Complexity Tables, Dunder Methods & OOP Hierarchies **5**
- 3.1 Asymptotic Complexity Analysis of Python Built-ins . . . 5
- 3.2 Dunder (Magic) Methods Reference Table . . . 5
- 3.3 Multiple Inheritance & C3 Linearization (MRO) . . . 6

4 Standard Library Architecture, Custom Exceptions & Import Mechanics **7**
- 4.1 Module Search Path Algorithm (`sys.path`) . . . 7
- 4.2 Python Exception Hierarchy Architecture . . . 7

5 NumPy Matrix Cheat-Sheet, Broadcasting Mechanics & Linear Algebra Methods **9**
- 5.1 NumPy Array Memory Layout & Strides . . . 9
- 5.2 Broadcasting Rules Matrix (The Two Invariants) . . . 9
- 5.3 `numpy.linalg` Essential Methods Reference . . . 9

6 File Modes, Pandas Data Wrangling & Matplotlib Visualization Cheat-Sheet **11**
- 6.1 File Open Modes & Buffering Reference . . . 11
- 6.2 Pandas Core Operations Cheat-Sheet . . . 11
- 6.3 Matplotlib Figure Architecture . . . 12

Chapter 1

Core Syntax, Types, Operators & Built-in Functions Reference

1.1 Python Virtual Machine Architecture & Memory Models

CPython Execution Pipeline

Pipeline Stages:

1. **Lexical Analysis & Parsing:** Source code (`.py`) is tokenized into an Abstract Syntax Tree (AST).
2. **Bytecode Compilation:** The AST is compiled into platform-independent CPython bytecode (`.pyc` stored in `__pycache__`).
3. **Evaluation Loop (PVM):** The Python Virtual Machine executes bytecode instructions via a massive switch statement in C (`ceval.c`).

CPython Object Model (`PyObject`): Every value in Python is a dynamically allocated C structure on the heap:

$$\text{PyObject} = \{\text{refcnt: 8 bytes}, \quad \text{type: 8 bytes}\} \tag{1.1}$$

For variable-length objects (e.g., `str`, `list`, `tuple`), `PyVarObject` adds an 8-byte `ob_size` header.

1.2 Primitive Data Types & Representation Limits

Type	Internal C Struct	Precision / Limits	Mutability & Key Semantics
`int`	`PyLongObject`	Arbitrary precision (memory-bound)	Immutable. Small integers $[-5, 256]$ are pre-allocated singletons in CPython.
`float`	`PyFloatObject`	64-bit IEEE 754 double precision	Immutable. 53-bit mantissa, 11-bit exponent, approx 15–17 decimal digits.
`complex`	`PyComplexObject`	Two 64-bit doubles (`real + imag*1j`)	Immutable. Conjugate accessible via `z.conjugate()`.
`bool`	`PyBoolObject`	Subclass of `int` (`True == 1, False == 0`)	Immutable singleton. Arithmetic allowed: `True + True == 2`.
`str`	`PyASCIIObject` / `PyCompactUnicode`	1, 2, or 4 bytes per char (PEP 393)	Immutable sequence of Unicode code points.
`bytes`	`PyBytesObject`	Raw 8-bit unsigned bytes ($[0, 255]$)	Immutable binary buffer.
`NoneType`	`_PyNone_Type`	Singleton object (`None`)	Truth value is always `False`.

Table 1.1: Primitive Python Built-in Data Types

1.3 Operator Precedence Matrix (Highest to Lowest)

Level	Operator	Associativity	Description & Architectural Notes
1	`(...)`, `[...]`, `{...}`	Left-to-right	Parentheses, list displays, dict/set comprehensions
2	`x[index]`, `x[i:j]`, `x(...)`, `x.attr`	Left-to-right	Subscripting, slicing, function call, attribute reference
3	`**`	**Right-to-left**	Exponentiation: `2 ** 3 ** 2 == 2 ** (3 ** 2) == 512`
4	`+x`, `-x`, `~x`	Right-to-left	Unary positive, unary negative, bitwise NOT (`~x == -(x+1)`)
5	`*`, `/`, `//`, `%`	Left-to-right	Multiplication, true float division, floor division, modulo
6	`+`, `-`	Left-to-right	Addition, subtraction (or sequence concatenation)
7	`«`, `»`	Left-to-right	Bitwise left shift, bitwise right shift
8	`&`	Left-to-right	Bitwise AND (set intersection)
9	`^`	Left-to-right	Bitwise XOR (symmetric difference)
10	`|`	Left-to-right	Bitwise OR (set union, dict merge in 3.9+)
11	`in`, `not in`, `is`, `is not`, `<`, `<=`, `>`, `>=`, `!=`, `==`	Chained	Comparison operators (chaining: `1 < x < 5` $\iff$ `1 < x and x < 5`)
12	`not x`	Right-to-left	Boolean NOT
13	`and`	Left-to-right	Short-circuit Boolean AND (evaluates second operand only if first is truthy)
14	`or`	Left-to-right	Short-circuit Boolean OR (evaluates second operand only if first is falsy)
15	`if - else`	Right-to-left	Conditional expression (Ternary operator: `x if C else y`)
16	`:=`	Right-to-left	Walrus assignment expression (PEP 572)

Table 1.2: Python Operator Precedence & Associativity

The Floor Division & Modulo Invariant:
In Python, integer division satisfies:

$$a = (a//b) \cdot b + (a \pmod b) \tag{1.2}$$

Because Python uses *flooring division* (towards $-\infty$) rather than truncation towards zero, `-7 // 3 == -3` and `-7 % 3 == 2`. The sign of `a % b` always matches the sign of divisor b!

Chapter 2

Control Flow, Iteration Protocols, Function Signatures & Regular Expressions

2.1 Control Flow Mechanics

Loop `else` Clause Invariant

In Python, both `for` and `while` loops accept an optional `else` block:

```
for item in iterable:
    if condition(item):
        break  # Skips loop else
else:
    # Executes ONLY if the loop completed without encountering a break
    handle_exhaustion()
```

Usage: Eliminates boolean flag variables when searching for an element in a collection.

2.2 Function Parameter Anatomy & Argument Unpacking

Complete Parameter Specification Hierarchy (PEP 3102 & PEP 570)

A fully generalized Python function signature is declared as:

```
def func(pos_only1, pos_only2, /, pos_or_kw, *args, kw_only1, kw_only2=val, **kwargs):
    pass
```

- `/` (Positional-only marker): Parameters preceding `/` cannot be passed as keyword arguments.
- `*args` (Variadic positional): Packs remaining positional arguments into an immutable `tuple`.
- **Keyword-only parameters**: Any parameter declared after `*` or `*args` MUST be passed by name.
- `**kwargs` (Variadic keyword): Packs extra keyword arguments into a mutable `dict`.

The Mutable Default Argument Trap:
Default arguments are evaluated **once at function definition time**, not at runtime!

```
# BUG: shared list between invocations
def append_to(element, target=[]):
    target.append(element)
    return target

# CORRECT: None sentinel idiom
def append_to(element, target=None):
    if target is None:
```

```
        target = []
    target.append(element)
    return target
```

2.3 Scope Resolution: The LEGB Rule

LEGB Search Hierarchy

Identifiers are resolved by inspecting nested namespaces in strict sequence:

$$\textbf{L}ocal \longrightarrow \textbf{E}nclosing \longrightarrow \textbf{G}lobal \longrightarrow \textbf{B}uilt-in \tag{2.1}$$

- `global var`: Rebinds variable in the module-level global namespace.
- `nonlocal var`: Rebinds variable in the nearest enclosing non-global scope (closures).

2.4 Regular Expressions Cheat-Sheet (`re` Module)

Pattern	Category	Semantics & Match Criteria
`\d` / `\D`	Character Class	Matches any Unicode digit ($[0-9]$) / non-digit
`\w` / `\W`	Character Class	Matches alphanumeric character plus underscore / non-alphanumeric
`\s` / `\S`	Character Class	Matches any whitespace char (`\t`, `\n`, space) / non-whitespace
`\b` / `\B`	Boundary Anchor	Word boundary (transition between `\w` and `\W`) / non-boundary
`^` / `$`	Line Anchors	Matches beginning of string (or line in MULTILINE) / end of string
`*`, `+`, `?`	Greedy Quantifiers	0 or more, 1 or more, 0 or 1 occurrences
`*?`, `+?`, `??`	Lazy Quantifiers	Non-greedy variants matching minimal characters necessary
`(?P<name>...)`	Named Group	Captures matched substring accessible by dictionary key in `match.groupdict()`
`(?:...)`	Non-capturing	Groups patterns for quantifiers without consuming memory in group tuples
`(?=...)` / `(?!...)`	Lookaround	Positive lookahead / negative lookahead (zero-width assertion)

Table 2.1: Essential Regular Expression Syntax

Chapter 3

Data Structure Complexity Tables, Dunder Methods & OOP Hierarchies

3.1 Asymptotic Complexity Analysis of Python Built-ins

Data Structure	Index / Key	Append / Add	Insert / Delete	Lookup	CPython Internal Structure
`list`	$\mathcal{O}(1)$	$\mathcal{O}(1)^*$ amortized	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Dynamic array of `PyObject*` pointers.
`tuple`	$\mathcal{O}(1)$	N/A (immutable)	N/A	$\mathcal{O}(n)$	Fixed-size array of `PyObject*` pointers.
`dict`	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ average	Combined hash table with sparse hash array + dense indices array (PEP 468).
`set`	N/A	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ average	Hash table with open addressing and perturbation collision resolution.
`collections.deque`	$\mathcal{O}(n)$	$\mathcal{O}(1)$ both ends	$\mathcal{O}(1)$ at ends	$\mathcal{O}(n)$	Doubly linked list of blocks (64 elements per node).

Table 3.1: CPython Core Data Structure Time Complexities (*amortized over reallocation)

List Growth Pattern in CPython

When a list exceeds its allocated capacity, CPython over-allocates according to:

$$\text{new_allocated} = \text{newsize} + (\text{newsize} \gg 3) + (\text{newsize} < 9 \ ? \ 3 : 6) \tag{3.1}$$

This geometric sequence $(0, 4, 8, 16, 24, 32, 40, 52, 64, \dots)$ ensures amortized $\mathcal{O}(1)$ appends while bounding memory waste to approximately 12.5%.

3.2 Dunder (Magic) Methods Reference Table

Category	Method Signature	Trigger Operation / Protocol
Lifecycle	`__new__(cls, *args)`	Static allocator that creates and returns new instance.
	`__init__(self, *args)`	Initializer called immediately after object creation.
	`__del__(self)`	Finalizer invoked when reference count reaches 0.
String / Repr	`__repr__(self)`	Unambiguous official representation for debugging (e.g. `eval(repr(x)) == x`).
	`__str__(self)`	User-friendly string representation (`print(x)`).
	`__bytes__(self)`	Invoked by `bytes(x)`.
Container	`__len__(self)`	Invoked by `len(x)` (must return non-negative int).
	`__getitem__(self, key)`	Subscript access `x[key]` or slice `x[i:j]`.
	`__setitem__(self, key, val)`	Subscript assignment `x[key] = val`.
	`__delitem__(self, key)`	Deletion `del x[key]`.
	`__contains__(self, item)`	Membership testing `item in x`.
Iteration	`__iter__(self)`	Returns iterator object implementing `__next__`.
	`__next__(self)`	Returns next item or raises `StopIteration`.
Context	`__enter__(self)`	Acquires resource for `with` statement.
	`__exit__(self, exc, val, tb)`	Cleans resource; suppresses exception if returns `True`.
Callable	`__call__(self, *args, **kw)`	Allows instance to be invoked as a function `obj()`.

Table 3.2: Essential Python Dunder Methods

3.3 Multiple Inheritance & C3 Linearization (MRO)

C3 Linearization Algorithm

For a class C with base classes $B_1, B_2, \ldots, B_n$, the Method Resolution Order $L(C)$ is:

$$L(C) = [C] + \text{merge}\Big(L(B_1), L(B_2), \ldots, L(B_n), [B_1, B_2, \ldots, B_n]\Big) \tag{3.2}$$

Merge Rule: Select the first head from candidate lists that does not appear in any candidate's tail. If found, extract that class, append to result, and repeat. If no head is valid, linearization fails with `TypeError: Cannot create a consistent method resolution order (MRO)`.

Chapter 4

Standard Library Architecture, Custom Exceptions & Import Mechanics

4.1 Module Search Path Algorithm (`sys.path`)

Import Resolution Sequence

When Python executes `import foo`:

1. **Cache Check:** Check `sys.modules['foo']`. If present, return cached module.
2. **Built-in Modules:** Check compiled built-in modules list (`sys.builtin_module_names`).
3. **Search `sys.path`:**
   - Directory containing the input script (or current working directory `""`).
   - `PYTHONPATH` environment variable directories.
   - Standard library directories (`lib/python3.x`).
   - Third-party site-packages directory (`site-packages`).
4. **Package Initialization:** If a directory containing `__init__.py` is found, initialize package.

4.2 Python Exception Hierarchy Architecture

Core Built-in Exception Tree (PEP 3151)

```
BaseException
 |-- SystemExit
 |-- KeyboardInterrupt
 |-- GeneratorExit
 |-- Exception (Catch all user-level exceptions here!)
      |-- ArithmeticError
      |    |-- ZeroDivisionError
      |    |-- OverflowError
      |    |-- FloatingPointError
      |-- AssertionError
      |-- AttributeError
      |-- BufferError
      |-- EOFError
      |-- ImportError
      |    |-- ModuleNotFoundError
      |-- LookupError
      |    |-- IndexError
      |    |-- KeyError
      |-- NameError
      |    |-- UnboundLocalError
      |-- OSError (EnvironmentError, IOError)
```

```
      |    |-- FileNotFoundError
      |    |-- PermissionError
      |    |-- TimeoutError
      |-- TypeError
      |-- ValueError
           |-- UnicodeError
```

Examiner Warning on Exception Traps:
Never write bare `except:`, and avoid `except BaseException:`. Catching `BaseException` intercepts `KeyboardInterrupt` (Ctrl+C) and `SystemExit`, preventing graceful program termination. Always subclass `Exception` for custom application exceptions:

```
class ApplicationError(Exception):
    '''Base exception for domain logic.'''
    pass
```

Chapter 5

NumPy Matrix Cheat-Sheet, Broadcasting Mechanics & Linear Algebra Methods

5.1 NumPy Array Memory Layout & Strides

`ndarray` Internal Anatomy

A NumPy array consists of a memory buffer and metadata:

- **Data Pointer:** Address of contiguous byte buffer in C heap.
- **Shape:** Tuple of dimension lengths $(d_0, d_1, \ldots, d_{k-1})$.
- **Strides:** Tuple of byte offsets to advance one element along each dimension:

$$\text{Address}(i, j) = \text{base} + i \cdot s_0 + j \cdot s_1 \tag{5.1}$$

- **Order:** C-contiguous (row-major: last index varies fastest) vs Fortran (column-major).

5.2 Broadcasting Rules Matrix (The Two Invariants)

NumPy Broadcasting Conditions

Two dimensions are compatible for element-wise operations if:

1. They are equal in size: $d_A = d_B$, OR
2. One of them is 1: $d_A = 1$ or $d_B = 1$ (the unit dimension is stretched without copying data).

Alignment Algorithm:

- Compare dimension shapes **from right to left** (trailing dimensions).
- Missing leading dimensions are prepended with size 1.

Example Compatibility:

$$\begin{aligned} &\text{Array A: } (256, 256, 3) \\ &\text{Array B: } (\qquad\quad 3) \quad \Longrightarrow \text{ Compatible! Result: } (256, 256, 3) \end{aligned}$$

5.3 `numpy.linalg` Essential Methods Reference

Function	Mathematical Equation	Operational Complexity & Purpose
`A @ B` / `np.matmul`	$C = AB$	Matrix product via BLAS DGEMM ($\mathcal{O}(n^3)$).
`np.linalg.inv(A)`	A^{-1} s.t. $AA^{-1} = I$	Exact matrix inversion via LU decomposition ($\mathcal{O}(n^3)$).
`np.linalg.pinv(A)`	$A^{+} = (A^T A)^{-1} A^T$	Moore-Penrose pseudo-inverse via SVD (least squares).
`np.linalg.det(A)`	$\det(A) = \prod \lambda_i$	Determinant calculated from LU diagonal ($\mathcal{O}(n^3)$).
`np.linalg.eig(A)`	$Av = \lambda v$	Full eigendecomposition of general square matrix.
`np.linalg.eigh(A)`	$Av = \lambda v$ $(A = A^H)$	Specialized faster LAPACK solver for symmetric/Hermitian matrices.
`np.linalg.svd(A)`	$A = U \Sigma V^T$	Singular Value Decomposition; basis for PCA and dimension reduction.
`np.linalg.cholesky(A)`	$A = LL^T$	Cholesky decomposition of symmetric positive-definite matrix ($\mathcal{O}(\frac{1}{3}n^3)$).
`np.linalg.solve(A, b)`	$Ax = b$	Solves linear system directly without computing explicit inverse.

Table 5.1: NumPy Linear Algebra Standard API

Chapter 6

File Modes, Pandas Data Wrangling & Matplotlib Visualization Cheat-Sheet

6.1 File Open Modes & Buffering Reference

Mode	Semantics	Stream Pointer Position & Behavior
`'r'`	Read text (default)	Pointer at start. Raises `FileNotFoundError` if missing.
`'w'`	Overwrite text	Truncates existing file to 0 bytes or creates new. Pointer at start.
`'a'`	Append text	Pointer at end. Creates file if missing. Writes always append.
`'r+'`	Read and Write	Pointer at start. Does not truncate file automatically.
`'w+'`	Overwrite and Read	Truncates file immediately to 0 bytes, then allows read and write.
`'a+'`	Append and Read	Pointer at end for writes. Initial read from end unless `seek(0)`.
`'rb'`, `'wb'`	Binary variants	Read/write raw byte buffers without Unicode encoding translation.

Table 6.1: Standard File Access Modes

6.2 Pandas Core Operations Cheat-Sheet

Operation	Syntax	Operational Semantics
Label Selection	`df.loc[row_lbl, col_lbl]`	Selects rows/columns by exact index labels (inclusive slices).
Integer Selection	`df.iloc[row_idx, col_idx]`	Selects rows/columns by integer positions $0 \dots N-1$ (exclusive end).
Boolean Filtering	`df[(df['A'] > 0) & (df['B'] == 'X')]`	Bitwise `&` and `|` with mandatory parentheses for condition grouping.
Missing Handling	`df.dropna(subset=['col'])`	Drops rows where specified columns contain NaN.
	`df['A'].fillna(df['A'].median())`	Imputes missing entries using median scalar.
Aggregation	`df.groupby('dept')['salary'].agg(['mean', 'std'])`	Split-apply-combine aggregation yielding multi-index columns.
Relational Merge	`pd.merge(df1, df2, on='id', how='left')`	SQL-style left join on key column.

Table 6.2: Pandas High-Yield Data Wrangling Operations

6.3 Matplotlib Figure Architecture

The Object-Oriented Hierarchy

Figure Canvas $\longrightarrow$ Figure $\longrightarrow$ Axes $\longrightarrow$ Artists (Lines, Text, Patches)

```
import matplotlib.pyplot as plt

# 1. Instantiate Figure and Axes grid
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(12, 5), dpi=300)

# 2. Plotting on Axes
ax1.plot(x, y, color='#0F2043', lw=2, label='Signal')
ax1.set_title('Time Domain Response')
ax1.legend(loc='best')
ax1.grid(True, linestyle='--', alpha=0.6)

ax2.hist(data, bins=30, color='#E85A10', edgecolor='black', alpha=0.7)
ax2.set_title('Frequency Distribution')

plt.tight_layout()
plt.savefig('output.pdf', bbox_inches='tight')
```